Yes do that. But also factor in all domains, subdomains, niche domains, and encompass all categorical asymmetries to the root and sub root and splinter root and splinter sub root and do that for all layer pattern hierarchies

Here is the complete **LexCore Data Fabric Neural Schema — Full Master Build**. All 140 nodes, all hierarchies, all asymmetries, ready for agent injection.

LexCore Complete Fabric Schema

All Domains · All Hierarchies · All Asymmetries

140 total nodes across 6 hierarchy levels, 4 law families, 24 splinter roots, 6 splinter sub-roots, 18 niche domains, 15 categorical asymmetry cross-connects, and 72 domain layer nodes.

The 6-Level Hierarchy

```
LLLL . DDDD . SSSS . NNNN . TTTT . AAAA   ← 24-bit signal
────   ────   ────   ────   ────   ────
 │      │      │      │      │      └── AXIS FLAG (V/H/D/TLTT/Q)
 │      │      │      │      └───────── TYPE FLAG
 │      │      │      └──────────────── NODE SEQUENCE
 │      │      └─────────────────────── SPLINTER/NICHE SEQ
 │      └────────────────────────────── DOMAIN CODE
 └───────────────────────────────────── LEVEL CODE
```

Level Code	Name	Count	Description
0000	Fabric Root	1	FR — LexCore universe master node
0001	Sub-Root	4	PuL · PrL · PcL · InL — Law families
0010	Splinter Root	24	All major domains (ConL, CrimL, IPL, CorpL…)
0011	Splinter Sub-Root	6	IP sub-domains (PL, CR, TM, TS, IndD, PBR)
0100	Niche	18	Deep specialist nodes (BioPat, WCC, SR&ED…)
1001	Asymmetry	15	Cross-domain intersections
0101–1000	Layer Nodes	72	V + H + TLTT per domain

Full Splinter Root Map

PUBLIC family — 7 splinter roots:

```
ConL → AdmL → CrimL → TaxL → ImmL → EnvL → SecL
```

PRIVATE family — 10 splinter roots:

```
CtL → TortL → PropL → CorpL → IPL → LabL → FamL → PrivL → InsL → EstL
```

PROCEDURAL family — 4 splinter roots:

```
CivPr → CrimPr → EvL → ClA
```

INTERNATIONAL family — 3 splinter roots:

```
IntTr → IntIP → IntArb
```

Categorical Asymmetries — The 15 Cross-Connects

These are **diagonal splice nodes** — they activate when a query crosses domain boundaries and require parallel agent dispatch:

Abbr	Domain 1 ⟺ Domain 2	Core Tension
`IPTax`	IP ⟺ TAX	SR&ED credits, patent box, IP transfer pricing
`IPCorp`	IP ⟺ CORP	IP assignment on incorporation, IP in M&A due diligence
`PrivIP`	PRIV ⟺ IP	Biometric data consent vs. AI patent development
`PrivCrim`	PRIV ⟺ CRIM	s.8 Charter search/seizure vs. cybercrime investigation
`CorpSec`	CORP ⟺ SEC	Public company continuous disclosure, insider trading
`TaxImm`	TAX ⟺ IMMIG	Deemed residency, departure tax, FATCA
`LabCrim`	LABOR ⟺ CRIM	Bill C-45 criminal workplace safety liability
`EnvCrim`	ENV ⟺ CRIM	CEPA criminal fines up to $12M/day
`ConAdm`	CONST ⟺ ADMIN	Charter challenges to regulations (Vavilov framework)
`LabImm`	LABOR ⟺ IMMIG	LMIA, TFWP, TN visa employment conditions
`CorpTax`	CORP ⟺ TAX	GAAR, thin cap, pipeline transactions, surplus stripping
`FamProp`	FAM ⟺ PROP	Matrimonial home, net family property equalization
`InsCorpD`	INS ⟺ CORP	D&O insurance, duty to defend, business judgment rule

Abbr	Domain 1 ⟺ Domain 2	Core Tension
`IPLab`	IP ⟺ LABOR	Employee-created IP ownership, non-compete + trade secrets
`IntlIP`	INTL ⟺ IP	PCT/Madrid/Berne/TRIPS multi-jurisdictional strategy

How Agents Navigate This Fabric

Every inbound query hits **LexRouter** first, which resolves the signal path in 4 steps:

```
# Step 1 — Level resolution (which hierarchy level does the query live at?)
level = resolve_level(query)     # NICHE | SPLINTER_SUB | SPLINTER_ROOT | ASYM

# Step 2 — Domain match (SPLICE_HEX fast identity lookup)
domain_hex = FABRIC_REGISTRY[abbr]["splice_hex"]

# Step 3 — Asymmetry check (does query cross domain boundaries?)
if cross_domain_detected(query):
    activate_diagonal(ASYMMETRIES[asym_key])  # parallel dual-domain search

# Step 4 — TLTT gate traversal (descend V → TLTT → V → TLTT → result)
result = traverse_tltt_spine(domain_hex, query_signal)
# Each TLTT is a hard gate — fail = route to LexDraft or rejection signal
# Pass = descend to next layer, collect structured output
# Terminal TLTT (H) = package result + inject disclaimer + return ⬆
```

Want me to generate the **agent injection code** — the Python registry that loads all 140 nodes at startup and exposes the full routing table to LexRouter?